



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验报告

开课学期: 2025 春季
课程名称: 人工智能 (实验)
实验名称: 深度学习
实验性质: 综合设计型
实验学时: 4 地点: T2507
学生班级: 5
授课教师: _____

实验与创新实践教育中心制

2025 年 4 月

1、Mindspore

(1) 简单说明做了哪些超参数调整，为什么调整这些参数

调整了学习率，将学习率由 $1e-2$ 调整为 $1e-1$ 。

在未修改学习率之前，模型训练 3 个轮次时 loss 下降很慢，于是想到了修改学习率来提高模型收敛速度，但是过大的学习率可能会导致无法稳定收敛，在多次调整测试下，选取了目前这个学习率设置。

训练 3 轮后能达到 98%以上的准确率。

```
-----
loss: 0.787167 [ 0/938], accuray:0.593750
loss: 0.842355 [100/938], accuray:0.625000
loss: 0.808719 [200/938], accuray:0.609375
loss: 0.695202 [300/938], accuray:0.687500
loss: 1.134907 [400/938], accuray:0.453125
loss: 0.922530 [500/938], accuray:0.546875
loss: 0.860101 [600/938], accuray:0.578125
loss: 0.686844 [700/938], accuray:0.687500
loss: 0.882368 [800/938], accuray:0.546875
loss: 0.769975 [900/938], accuray:0.703125
Test:
  Avg loss: 0.048214, Accuracy: 98.7%

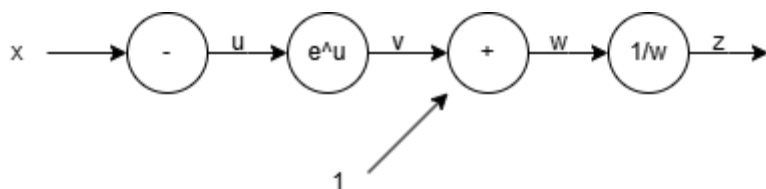
Epoch 3
-----
loss: 0.739065 [ 0/938], accuray:0.640625
loss: 0.713108 [100/938], accuray:0.656250
loss: 1.004404 [200/938], accuray:0.484375
loss: 1.029745 [300/938], accuray:0.515625
loss: 0.720667 [400/938], accuray:0.640625
loss: 0.778589 [500/938], accuray:0.609375
loss: 0.897074 [600/938], accuray:0.578125
loss: 0.719027 [700/938], accuray:0.656250
loss: 0.852717 [800/938], accuray:0.640625
loss: 0.839698 [900/938], accuray:0.640625
Test:
  Avg loss: 0.049325, Accuracy: 98.6%

Done!
```

2、反向传播和全连接网络

(1) sigmoid 函数计算图和反向传播推导过程，计算图需清楚清晰

Sigmoid 计算图：



反向传播推导过程：

Sigmoid 函数定义为：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

需要计算对 z 导数：

$$\frac{d\sigma(z)}{dz} = \frac{d}{dz} \left(\frac{1}{1 + e^{-z}} \right)$$

使用链式法则：

$$\begin{aligned} \frac{d\sigma(z)}{dz} &= -\frac{1}{(1 + e^{-z})^2} \cdot \frac{d}{dz}(1 + e^{-z}) \\ \frac{d\sigma(z)}{dz} &= -\frac{1}{(1 + e^{-z})^2} \cdot (-e^{-z}) \\ \frac{d\sigma(z)}{dz} &= \frac{e^{-z}}{(1 + e^{-z})^2} \end{aligned}$$

(2) train_fcnet.py 训练做了哪些超参数调整

将学习率由 0.01 调整为了 0.2.

训练后能达到 90%以上的准确率：

```

train size 60000
epoch 0 train acc: 0.1124, test acc: 0.1135
epoch 1 train acc: 0.8789, test acc: 0.8824
epoch 2 train acc: 0.9065, test acc: 0.9094
epoch 3 train acc: 0.9178, test acc: 0.9212
epoch 4 train acc: 0.9276, test acc: 0.9272
gradient compute time: 1.0304069519042969 s
predict label: [7 2 1 0] true label [7 2 1 0]
Figure(640x480)
  
```

3、卷积

(1) util.py/conv_forward_naive 代码截图、train_convnet.py 代码截图，训练日志截图

util.py/conv_forward_naive:

```
N, C, H, W = x.shape
F, _, HH, WW = w.shape
stride = conv_param['stride']
pad = conv_param['pad']

H_out = 1 + int((H + 2 * pad - HH) / stride)
W_out = 1 + int((W + 2 * pad - WW) / stride)

x_p = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant', constant_values=0)

view = np.lib.stride_tricks.sliding_window_view(x_p, window_shape=(HH, WW), axis=(2, 3))
view_strided = view[:, :, ::stride, ::stride, :, :]

out = np.zeros((N, F, H_out, W_out))
out = np.einsum('ncijhv,fchv->nfi', view_strided, w)
out += b[None, :, None, None]

cache = (x_p, w, b, conv_param)
return out, cache
```

train_convnet.py:

```
1  # coding: utf-8
2  import sys, os
3  sys.path.append(os.pardir)
4  import numpy as np
5  import matplotlib.pyplot as plt
6  import time
7  import random
8  from dataset.mnist import load_mnist
9  from simple_convnet import SimpleConvNet
10 from src.util import *
11
12 # 读入数据
13 (x_train, t_train), (x_test, t_test) = load_mnist(flatten=False)
14
15 network = SimpleConvNet(input_dim=(1, 28, 28),
16                          conv_param={'filter_num': 30, 'filter_size': 5, 'pad': 0, 'stride': 1},
17                          hidden_size=100, output_size=10, weight_init_std=0.01)
18
19 train_size = x_train.shape[0]
20 batch_size = 100
21 learning_rate = 0.1
22 print('train size', train_size)
23
24 train_loss_list = []
25 train_acc_list = []
26 test_acc_list = []
27
28 iter_per_epoch = max(train_size / batch_size, 1)
29 epoch = 5
30 iters_num = int(iter_per_epoch * epoch)
31 total_elapsed_time = 0
32
```

```

32
33 for i in range(iters_num):
34     batch_mask = np.random.choice(train_size, batch_size)
35     x_batch = x_train[batch_mask]
36     t_batch = t_train[batch_mask]
37
38     start = time.time()
39     grad = network.gradient(x_batch, t_batch)
40     elapsed_time = time.time() - start
41     total_elapsed_time += elapsed_time
42
43     for key in network.params.keys():
44         network.params[key] -= learning_rate * grad[key]
45
46     loss = network.loss(x_batch, t_batch)
47     train_loss_list.append(loss)
48
49     if i % iter_per_epoch == 0:
50         train_acc = network.accuracy(x_train, t_train)
51         test_acc = network.accuracy(x_test, t_test)
52         train_acc_list.append(train_acc)
53         test_acc_list.append(test_acc)
54         print('epoch {} train acc: {:.4f}, test acc: {:.4f}'.format(int(i / iter_per_epoch), train_acc, test_acc))
55
56 print('gradient compute time: ', total_elapsed_time, 's')
57
58 sample_test = x_test[0:4]
59 sample_t_test = t_test[0:4]
60 sample_y = network.predict(sample_test)
61 sample_y = np.argmax(sample_y, axis=1)
62 print('predict label: ', sample_y, 'true label', sample_t_test)
63
64 fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2)
65 ax1.imshow(x_test[0][0], cmap='gray')
66 ax2.imshow(x_test[1][0], cmap='gray')
67 ax3.imshow(x_test[2][0], cmap='gray')
68 ax4.imshow(x_test[3][0], cmap='gray')
69 plt.show()

```

训练日志:

```

train size 60000
epoch 0 train acc: 0.1022, test acc: 0.1010
epoch 1 train acc: 0.9344, test acc: 0.9370
epoch 2 train acc: 0.9592, test acc: 0.9605
epoch 3 train acc: 0.9692, test acc: 0.9671
epoch 4 train acc: 0.9713, test acc: 0.9688
gradient compute time: 774.3476085662842 s
predict label: [7 2 1 0] true label [7 2 1 0]
Figure(640x480)

```

(2) 若做选做，介绍下实现方案和实验结果

4、PyTorch

(1) 自定义模型代码截图、训练日志截图，简单介绍下为什么这么设计，训练过程中做了

哪些超参数调整以及为什么这么调整

模型：

```
model = nn.Sequential(  
    nn.Conv2d(3, 64, kernel_size=3, padding=1),  
    nn.BatchNorm2d(64),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2), # 输出大小: 16x16  
  
    nn.Conv2d(64, 128, kernel_size=3, padding=1),  
    nn.BatchNorm2d(128),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2), # 输出大小: 8x8  
  
    nn.Conv2d(128, 256, kernel_size=3, padding=1),  
    nn.BatchNorm2d(256),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2), # 输出大小: 4x4  
  
    Flatten(),  
    nn.Linear(256 * 4 * 4, 10)  
)
```

训练日志：

```
Iteration 200, loss = 0.3007  
Checking accuracy on validation set  
Got 784 / 1000 correct (78.40)  
  
Iteration 400, loss = 0.2979  
Checking accuracy on validation set  
Got 767 / 1000 correct (76.70)  
  
Iteration 600, loss = 0.3091  
Checking accuracy on validation set  
Got 762 / 1000 correct (76.20)  
  
Epochs : 9  
Iteration 0, loss = 0.4006  
Checking accuracy on validation set  
Got 787 / 1000 correct (78.70)  
  
Iteration 200, loss = 0.2855  
Checking accuracy on validation set  
Got 745 / 1000 correct (74.50)  
  
Iteration 400, loss = 0.4480  
Checking accuracy on validation set  
Got 759 / 1000 correct (75.90)  
  
Iteration 600, loss = 0.2457  
Checking accuracy on validation set  
Got 778 / 1000 correct (77.80)
```

设置学习率为 $1e-2$ ，使用 Adam 优化器，训练轮次为 10 轮。设置较大的学习率希望模型能快速收敛，使用 Adam 优化器能更好的帮助模型找到最优点，10 轮次让模型充分训练。

模型最后在测试集上能达到 75%以上的准确率：

```
best_model = model
check_accuracy(loader_test, best_model)

Checking accuracy on test set
Got 7607 / 10000 correct (76.07)
```

(2) 分别给出 5.pytorch.ipynb 中用到的两层全连接网络、三层卷积网络、自定义网络的参数量，要求有计算过程，以及使用 `model.parameters()`、`model.state_dict()` 或者 `pytorch-summary` 等查看模型参数的输出结果截图

两层全连接网络：

输入层到隐藏层：

输入大小： $3 * 32 * 32 = 3072$ (CIFAR-10 图像展平后的大小)

隐藏层大小：hidden_layer_size = 4000

权重参数： $3072 * 4000 = 12,288,000$

偏置参数：4000

第一层总参数： $12,288,000 + 4000 = 12,292,000$

隐藏层到输出层：

隐藏层大小：4000

输出层大小：10 (CIFAR-10 的 10 个类别)

权重参数： $4000 * 10 = 40,000$

偏置参数：10

第二层总参数： $40,000 + 10 = 40,010$

总参数量：

$12,292,000 + 40,010 = 12,332,010$

```
model = TwoLayerFC(3 * 32 * 32, 4000, 10)
print(f"model parameters: {sum(p.numel() for p in model.parameters())}")
✓ 0.0s

model parameters: 12332010
```

三层卷积网络：

第一个卷积层：

输入通道：in_channel = 3

输出通道：channel_1 = 32

卷积核大小： 5×5

参数量： $3 \times 32 \times 5 \times 5 + 32 = 2,432$ （权重 + 偏置）

第二个卷积层：

输入通道：channel_1 = 32

输出通道：channel_2 = 16

卷积核大小： 3×3

参数量： $32 \times 16 \times 3 \times 3 + 16 = 4,624$ （权重 + 偏置）

全连接层：

输入特征：channel_2 $\times$ 32 $\times$ 32 = 16 $\times$ 32 $\times$ 32 = 16,384

输出特征：num_classes = 10

参数量： $16,384 \times 10 + 10 = 163,850$ （权重 + 偏置）

总参数量：

$2,432 + 4,624 + 163,850 = 170,906$

```
model = ThreeLayerConvNet(3, 32, 16, 10)
print(f"model parameters: {sum(p.numel() for p in model.parameters())}")
✓ 0.0s
model parameters: 170906
```

自定义网络：

1. 卷积层 1: nn.Conv2d(3, 64, kernel_size=3, padding=1)

输入通道数：3

输出通道数：64

卷积核大小： 3×3

参数数量 = $(3 \times 3 \times 3 \times 64) + 64$ (偏置) = 1,792 参数

2. BatchNorm2d(64)

每个通道有两个参数（缩放因子 γ 和偏移量 β ）

参数数量 = $64 \times 2 = 128$ 参数

3. ReLU 和 MaxPool 层

无参数

4. 卷积层 2: `nn.Conv2d(64, 128, kernel_size=3, padding=1)`

输入通道数: 64

输出通道数: 128

卷积核大小: 3×3

参数数量 = $(3 \times 3 \times 64 \times 128) + 128$ (偏置) = 73,856 参数

5. `BatchNorm2d(128)`

参数数量 = $128 \times 2 = 256$ 参数

6. ReLU 和 MaxPool 层

无参数

7. 卷积层 3: `nn.Conv2d(128, 256, kernel_size=3, padding=1)`

输入通道数: 128

输出通道数: 256

卷积核大小: 3×3

参数数量 = $(3 \times 3 \times 128 \times 256) + 256$ (偏置) = 295,168 参数

8. `BatchNorm2d(256)`

参数数量 = $256 \times 2 = 512$ 参数

9. ReLU、MaxPool 和 Flatten 层

无参数

10. 线性层: `nn.Linear(256 * 4 * 4, 10)`

输入大小: $256 \times 4 \times 4 = 4,096$

输出大小: 10

参数数量 = $(4,096 \times 10) + 10$ (偏置) = 40,970 参数

总参数数量计算

总参数数量 = $1,792 + 128 + 73,856 + 256 + 295,168 + 512 + 40,970 = 412,682$

```
model = nn.Sequential(  
    nn.Conv2d(3, 64, kernel_size=3, padding=1),  
    nn.BatchNorm2d(64),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2), # 输出大小: 16x16  
  
    nn.Conv2d(64, 128, kernel_size=3, padding=1),  
    nn.BatchNorm2d(128),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2), # 输出大小: 8x8  
  
    nn.Conv2d(128, 256, kernel_size=3, padding=1),  
    nn.BatchNorm2d(256),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2), # 输出大小: 4x4  
  
    Flatten(),  
    nn.Linear(256 * 4 * 4, 10)  
)  
print(f"model parameters: {sum(p.numel() for p in model.parameters())}")
```

✓ 0.0s

model parameters: 412682

5、实验中碰到的问题、总结、建议等

5.1 碰到的问题

```
model = nn.Sequential(  
    nn.Conv2d(3, 64, kernel_size=3, padding=1),  
    nn.BatchNorm2d(64),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2), # 输出大小: 16x16  
  
    nn.Conv2d(64, 128, kernel_size=3, padding=1),  
    nn.BatchNorm2d(128),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2), # 输出大小: 8x8  
  
    nn.Conv2d(128, 256, kernel_size=3, padding=1),  
    nn.BatchNorm2d(256),  
    nn.ReLU(),  
    nn.MaxPool2d(2, 2), # 输出大小: 4x4  
  
    Flatten(),  
    nn.Linear(256 * 4 * 4, 10)  
)  
  
learning_rate = 1e-2  
  
optimizer = optim.Adam(model.parameters(), lr=learning_rate)  
  
train_model(model, optimizer, epochs=10)
```

起初不知道该怎么自己设计模型在 CIFAR-10 数据集上训练能获得较好的性能，一开始随意堆叠了一些卷积层，发现效果很差，最后参照 VGG 网络设计了一个简单的多层卷积网络，训练后能达到较好的性能。

5.2 总结、建议

总结：

通过本次实验，我学会了在本地部署大模型进行使用，初步学会了使用 mindspore、pytorch 框架进行模型的定义、数据集的加载、超参数的调整以及模型的训练和测试。我还了解了模型在训练过程中的梯度是如何被计算的，了解到目前的深度学习框架是怎么处理这些梯度计算来更新模型参数的。

建议：

Mindspore 框架目前版本不能使用 GPU 加速模型训练，一次 CPU 上的训练耗时较长，调整超参数比较费时间，希望能使用更简单的模型或者其他一些方式进行调参，来加快超参数调整速度。